LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

STACK DAN QUEUE

Oleh:

Nurfitria Ramadhani NIM. 2510817220023

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL

2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Nurfitria Ramadhani
NIM : 2510817220023

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIM. 2310817310009	NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1 1
A. Source Code 1
B. Pembahasan 2
SOAL 2 4
A. Source Code 5
B. Output Program 7
C. Pembahasan 7
SOAL 3 9
A. Source Code 9
B. Pembahasan 11
SOAL 4 13
A. Source Code 14
B. Output Program 16
C. Pembahasan 16
TAUTAN GIT 18

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 2A 7
Gambar 2. Screenshot Hasil Jawaban Soal 2B 7
Gambar 3. Screenshot Hasil Jawaban Soal 2C 7
Gambar 4. Screenshot Hasil Jawaban Soal 4A 16
Gambar 5. Screenshot Hasil Jawaban Soal 4B 16
Gambar 6. Screenshot Hasil Jawaban Soal 4C 16

DAFTAR TABEL

Tabel 1. Source Code stack.cpp 1
Tabel 2. Source Code prob1.cpp 5
Tabel 3. Source Code queue.cpp 9
Tabel 4. Source Code prob2.cpp 14

SOAL 1

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1. Source Code stack.cpp

```
#include <iostream>
#include "stack.h"

void init(Stack* s) {
    s->top = &(s->data[-1]);
}

bool isEmpty(const Stack* s) {
    return s->top == &(s->data[-1]);
}

bool isFull(const Stack* s) {
    return s->top == &(s->data[MAX - 1]);
}

void push(Stack* s, int value) {
    if (isFull(s)) {
```

```
        throw "Stack Overflow";
    }
    s->top++;
    *(s->top) = value;
}

void pop(Stack* s) {
    if (!isEmpty(s)) {
        s->top--;
    }
}

int peek(const Stack* s) {
    return *(s->top);
}
```

B. Pembahasan

`#include <iostream> & "stack.h"`: Digunakan untuk memanggil pustaka standar input-output dan menghubungkan implementasi dengan definisi struktur data pada file header stack.h.

`void init(Stack* s)`: Berfungsi untuk menginisialisasi stack dengan mengarahkan *pointer* top ke indeks sebelum elemen pertama (-1) sebagai tanda stack masih kosong.

`bool isEmpty(const Stack* s)`: Digunakan untuk memeriksa apakah stack kosong dengan membandingkan posisi top saat ini dengan alamat indeks -1.

`bool isFull(const Stack* s)`: Berfungsi untuk memeriksa apakah stack sudah penuh dengan mengecek apakah top sudah mencapai batas maksimal indeks (MAX - 1).

`void push(Stack* s, int value)`: Digunakan untuk memasukkan data baru ke dalam stack; posisi top akan naik satu langkah (top++) lalu diisi dengan nilai baru.

`throw "Stack Overflow"`: Berfungsi sebagai penanganan kesalahan untuk menghentikan program jika pengguna mencoba memasukkan data ke dalam stack yang sudah penuh.

`void pop(Stack* s)`: Berfungsi untuk menghapus elemen teratas stack cukup dengan menurunkan posisi *pointer* top ke indeks di bawahnya.

`int peek(const Stack* s)`: Digunakan untuk mengambil atau melihat nilai yang berada di posisi top saat ini tanpa menghapusnya dari stack.

SOAL 2

Time Limit: 1.0 s

Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- Ai adalah nilai bilangan pada input

- Simbol yang valid hanya:
  - bilangan bulat
  - operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang symbol
- Baris kedua berisi simbol A sebanyak n buah Format Output Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	**Output**
3 2 3 +	5
9 5 1 2 + 4 * + 3	14
5 10 2 / 3 *	15

A. Source Code

Tabel 2. Source Code prob1.cpp

```
#include <iostream>
#include <string>
#include "stack.h"

using namespace std;

```

```
int main() {
    Stack s;
    init(&s);

    int n;
    cin >> n;

    for (int i = 0; i < n; i++) {
        string simbol;
        cin >> simbol;

        if (simbol == "+" || simbol == "-" || simbol == "*" || simbol == "/") {
            int bil2 = peek(&s); pop(&s);
            int bil1 = peek(&s); pop(&s);

            if (simbol == "+") push(&s, bil1 + bil2);
            else if (simbol == "-") push(&s, bil1 - bil2);
            else if (simbol == "*") push(&s, bil1 * bil2);
            else if (simbol == "/") push(&s, bil1 / bil2);
        }
        else {
            int angka = stoi(simbol);
            push(&s, angka);
        }
    }

    cout << peek(&s) << endl;

    return 0;
}
```

B. Output Program

```
PS C:\Users\ThinkPad\Desktop\Kuliah\Semester 2\Praktikum Algo\Modul 2\module-2-stack-and-queue-sapimouuu-main> ./hasil
3
2 3 +
5
```

Gambar 1. Screenshot Hasil Jawaban Soal 2A

```
PS C:\Users\ThinkPad\Desktop\Kuliah\Semester 2\Praktikum Algo\Modul 2\module-2-stack-and-queue-sapimouuu-main> ./hasil
9
5 1 2 + 4 * + 3 -
14
```

Gambar 2. Screenshot Hasil Jawaban Soal 2B

```
PS C:\Users\ThinkPad\Desktop\Kuliah\Semester 2\Praktikum Algo\Modul 2\module-2-stack-and-queue-sapimouuu-main> ./hasil
5
10 2 / 3 *
15
```

Gambar 3. Screenshot Hasil Jawaban Soal 2C

C. Pembahasan

`#include <iostream>, <string>, "stack.h"`: Digunakan untuk memanggil library standar input-output, manipulasi string, dan file header kustom stack.h untuk operasi stack.

`int main()`: Berfungsi sebagai titik awal atau fungsi utama eksekusi program.

`Stack s; init(&s);`: Digunakan untuk mendeklarasikan variabel s dengan tipe data Stack dan menginisialisasi kondisi awal stack tersebut.

`int n; cin >> n;`: Berfungsi untuk mendeklarasikan dan mengambil input jumlah elemen atau simbol yang akan diproses.

`for (int i = 0; i < n; i++)`: Merupakan perulangan untuk memproses setiap simbol input sebanyak n kali.

`string simbol; cin >> simbol;`: Digunakan untuk menerima input berupa string yang bisa berupa angka atau operator aritmatika.

`if (simbol == "+" || ...)`: Kondisi untuk mengecek apakah simbol yang dimasukkan adalah operator aritmatika (+, -, *, /).

`int bil2 = peek(&s); pop(&s);`: Digunakan untuk mengambil nilai teratas stack (operand kedua) dan menghapusnya dari stack.

`int bil1 = peek(&s); pop(&s);`: Digunakan untuk mengambil nilai teratas berikutnya (operand pertama) dan menghapusnya dari stack.

`push(&s, bil1 [operator] bil2);`: Berfungsi untuk melakukan operasi aritmatika sesuai simbol dan memasukkan hasilnya kembali ke dalam stack.

`else { int angka = stoi(simbol); push(&s, angka); }`: Jika input bukan operator, string dikonversi menjadi integer menggunakan stoi dan dimasukkan ke dalam stack.

`cout << peek(&s) << endl;`: Berfungsi untuk menampilkan hasil akhir perhitungan yang tersisa di puncak stack ke layar.

`return 0;`: Berfungsi untuk mengembalikan nilai 0 ke sistem sebagai tanda bahwa program telah berakhir dengan sukses.

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3. Source Code queue.cpp

```
#include "queue.h"
#include <iostream>
#include <stdexcept>

using namespace std;

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return q->front == nullptr;
}

bool isFull(const Queue* q) {
    if (isEmpty(q)) return false;
    return q->rear == &(q->data[MAX - 1]);
```

```
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
        throw runtime_error("Queue Overflow");
    }

    if (isEmpty(q)) {
        q->data[0] = value;
        q->front = &(q->data[0]);
        q->rear = &(q->data[0]);
    } else {
        q->rear++;
        *(q->rear) = value;
    }
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
        throw runtime_error("Queue Underflow");
    }

    if (q->front == q->rear) {
        q->front = nullptr;
        q->rear = nullptr;
    } else {
        q->front++;
    }
}

int front(const Queue* q) {
```

```
    if (isEmpty(q)) {
        throw runtime_error("Queue is Empty");
    }
    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) {
        throw runtime_error("Queue is Empty");
    }
    return *(q->rear);
}
```

B. Pembahasan

`#include "queue.h" & <iostream>`: Digunakan untuk menghubungkan kode dengan definisi struktur data Queue dan memanggil library standar input-output.

`void init(Queue q)*`: Berfungsi untuk menginisialisasi antrean dengan mengatur *pointer* front dan rear ke nullptr sebagai tanda antrean masih kosong.

`bool isEmpty(const Queue q)*`: Digunakan untuk memeriksa apakah antrean kosong dengan mengecek apakah *pointer* front tidak menunjuk ke alamat mana pun (nullptr).

`bool isFull(const Queue q)*`: Berfungsi untuk memeriksa apakah antrean sudah penuh dengan mengecek apakah *pointer* rear sudah menunjuk ke alamat indeks terakhir dari array data.

`void enqueue(Queue q, int value)*`: Digunakan untuk menambahkan elemen baru ke bagian belakang antrean; jika kosong, elemen diletakkan di indeks 0, jika tidak, *pointer* rear akan bergeser naik.

`throw runtime_error`: Berfungsi untuk melemparkan pesan kesalahan jika terjadi kondisi *Overflow* (antrean penuh) atau *Underflow* (antrean kosong) agar program tidak berjalan tidak normal.

`void dequeue(Queue q)*`: Digunakan untuk menghapus elemen dari bagian depan antrean dengan menggeser *pointer* front naik satu langkah.

`int front(const Queue q)*`: Berfungsi untuk mengambil atau melihat nilai elemen yang berada di posisi paling depan dalam antrean.

`int back(const Queue q)*`: Berfungsi untuk mengambil atau melihat nilai elemen yang berada di posisi paling belakang dalam antrean.

SOAL 4

Time Limit: 1.0 s

Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- Ai: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk

- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi.

Contoh Kasus

Input	**Output**
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -2 2 6

A. Source Code

Tabel 4. Source Code prob2.cpp

```
#include <iostream>
#include <stdexcept>
#include "queue.h"

using namespace std;

int main() {
    int n, k;
    if (!(cin >> n >> k)) return 0;

    int arr[105];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

```

```
    if (k <= 0 || k > n) return 0;

    Queue q;
    init(&q);

    long long currentSum = 0;
    try {
        for (int i = 0; i < k; i++) {
            enqueue(&q, arr[i]);
            currentSum += arr[i];
        }
        cout << currentSum;

        for (int i = k; i < n; i++) {
            int keluar = front(&q);
            dequeue(&q);

            enqueue(&q, arr[i]);
            currentSum = currentSum - keluar + arr[i];
            cout << " " << currentSum;
        }
        cout << endl;
    } catch (...) {
        return 0;
    }
    return 0;
}
```

B. Output Program

```
PS C:\Users\ThinkPad\Desktop\Kuliah\Semester 2\Praktikum Algo\Modul 2\module-2-stack-and-queue-sapimouuu-main> ./hasil
5 3
1 2 3 4 5
6 9 12
```

Gambar 4. Screenshot Hasil Jawaban Soal 4A

```
PS C:\Users\ThinkPad\Desktop\Kuliah\Semester 2\Praktikum Algo\Modul 2\module-2-stack-and-queue-sapimouuu-main> ./hasil
5 5
1 2 3 4 5
15
```

Gambar 5. Screenshot Hasil Jawaban Soal 4B

```
PS C:\Users\ThinkPad\Desktop\Kuliah\Semester 2\Praktikum Algo\Modul 2\module-2-stack-and-queue-sapimouuu-main> ./hasil
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
```

Gambar 6. Screenshot Hasil Jawaban Soal 4C

C. Pembahasan

`#include <iostream>, <stdexcept>, & "queue.h":` Digunakan untuk memanggil library input-output, penanganan error standar, dan file header queue.h untuk operasi antrean.

`int n, k; cin >> n >> k;`: Berfungsi untuk mengambil input jumlah total elemen (n) dan ukuran jendela/window (k) yang akan dihitung.

`int arr[105];`: Digunakan untuk mendeklarasikan array statis guna menyimpan sekumpulan bilangan bulat dari input pengguna.

`Queue q; init(&q);`: Berfungsi untuk mendeklarasikan variabel antrean q dan menginisialisasi kondisi awalnya menggunakan fungsi init.

`long long currentSum = 0;`: Digunakan untuk menyimpan hasil penjumlahan elemen di dalam jendela saat ini dengan tipe data yang mampu menampung nilai besar.

`for (int i = 0; i < k; i++)`: Merupakan perulangan pertama untuk mengisi antrean dengan elemen awal sebanyak ukuran k dan menghitung jumlah total awalnya.

`enqueue(&q, arr[i]);`: Berfungsi untuk memasukkan elemen array ke dalam antrean sebagai bagian dari jendela yang sedang diproses.

`for (int i = k; i < n; i++)`: Merupakan perulangan kedua untuk menggeser jendela (sliding) dari elemen ke-k hingga elemen terakhir.

`int keluar = front(&q); dequeue(&q);`: Digunakan untuk mengambil nilai terdepan antrean yang akan keluar dari jendela, lalu menghapusnya menggunakan dequeue.

`currentSum = currentSum - keluar + arr[i];`: Berfungsi untuk memperbarui total penjumlahan dengan cara mengurangi elemen yang keluar dan menambah elemen baru yang masuk ke jendela.

`cout << " " << currentSum;`: Digunakan untuk menampilkan hasil penjumlahan setiap pergeseran jendela ke layar secara berurutan.

`catch (...) { return 0; }`: Berfungsi sebagai blok pengaman untuk menangkap segala jenis exception atau error agar program dapat berhenti secara normal tanpa crash.

`return 0;`: Berfungsi untuk mengembalikan nilai 0 kepada sistem sebagai tanda bahwa seluruh tahapan eksekusi program berjalan dengan sukses.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-sapimouuu